General - RPS Data

NIIT TRAINING DIVISION | LTM - iRUN Full Stack Engineering

TRAINING LAB GUIDE

LAB EXERCISE - 4

Consumer Groups, Rebalancing & Consumer Lag

Configure a Consumer Group · Observe Rebalancing · Generate and Drain Lag · Reset Offsets Safely

Module Series	Apache Kafka Operations & Streaming Platforms — Day 4
Lab Type	Guided Hands-On SOP — Kafka Consumer Operations
Duration	48 minutes (5 min setup + 25 min tasks + 10 min validation + 8 min debrief)
Difficulty	⭐⭐⭐ Beginner-Intermediate — Guided Multi-Terminal Exercise
Platform	Linux lab VM with Docker Engine + Compose v2; Apache Kafka 4.3.1 (apache/kafka image), 3-node KRaft cluster
Document Version	v1.0 | IT Training Division | 2026
Learner Name	_____
Date & Batch	_____
Prerequisite Lab Completed	☐ Day 4 Lab 3 (Brokers, Replication & Delivery Guarantees) — signed off; all 3 brokers running
Instructor Sign-Off	_____

1. LAB OVERVIEW

1.1 Why This Lab Exists

Consumers do the work in Kafka, and consumer groups are how that work is shared: each partition of a topic is assigned to exactly one member of the group, and every group remembers its position (committed offset) in each partition. When members join or leave, the group rebalances. When consumers fall behind producers, lag grows. Consumer lag is the single most important health metric of a streaming pipeline. In this lab you build a consumer group across three terminals, watch it rebalance, deliberately create lag, drain it, and reset offsets to replay data — the everyday operations of a Kafka support engineer.

NIIT iRUN Alignment

INCIDENT OWNERSHIP: The billing service is "running but behind". You own the diagnosis: how many messages behind, on which partitions, which consumer owns them — and whether it is catching up.

TROUBLESHOOTING EXCELLENCE: You will read kafka-consumer-groups output (CURRENT-OFFSET, LOG-END-OFFSET, LAG, CONSUMER-ID, STATE) and explain each column during a live rebalance.

OPERATIONAL READINESS: Offset resets are done with --dry-run first, on a stopped group, with the business impact (replay = duplicates, skip = loss) stated in advance.

1.2 Lab Scenario

Incident Briefing — Billing Is Behind After a Deployment

TIME: 13:30 | PRIORITY: HIGH | ASSIGNED TO: You

REQUEST RECEIVED: "The billing consumers were restarted during a deployment. Invoices are now minutes late. Show us how the group shares work, what happens when instances restart, how far behind we are, and how we could replay yesterday's events if billing finds a bug."

CONTEXT: You will use a new topic events (6 partitions) and a consumer group billing made of console consumers in three terminals.

YOUR MISSION: Configure the group, observe three rebalances, generate and drain lag, and replay with an offset reset. You have 25 minutes for the core tasks.

Figure: Lab 4 — Starting Environment: Healthy 3-Broker Cluster, No events Topic, No billing Group

1.3 Conceptual Foundation

Four ideas explain everything you will see in the consumer-groups tool.

Figure: Consumer Group — Partitions Divided Among Members · Committed Offset per Partition · Lag = Log-End Offset − Committed Offset

General - RPS Data

Concept	Meaning	Column / Command	Rule of Thumb
Group & members	Consumers sharing one group.id split the partitions between them	CONSUMER-ID, #MEMBERS	Max useful members = number of partitions
Committed offset	The next offset the group will read in each partition	CURRENT-OFFSET	Stored in __consumer_offsets; survives restarts
Lag	How many messages the group is behind in a partition	LAG = LOG-END-OFFSET − CURRENT-OFFSET	Growing lag = consumers slower than producers
Rebalance	Redistribution of partitions when members join, leave or crash	STATE: PreparingRebalance → Stable	Brief pause in consumption during the change

Figure: Rebalance Timeline — 1 Member (6 Partitions) → 2 Members (3 + 3) → 3 Members (2 + 2 + 2) → Member Leaves → 2 Members (3 + 3)

1.4 Learning Objectives

- Configure a consumer group and read its offsets, lag and members with kafka-consumer-groups
- Observe rebalancing as consumers join and leave, and explain partition assignment
- Generate consumer lag deliberately, watch it grow and drain, and interpret the LAG column
- Reset a group's offsets safely with --dry-run and --execute to replay data
- Explain the relationship between commits, rebalances and at-least-once delivery
- Verify each step using documented evidence: member counts, assignments, lag totals and new offsets

1.5 What You Will NOT Do

Hard Stops — Absolute Prohibitions for This Lab

✗ NEVER reset offsets without running --dry-run first

✗ NEVER reset offsets on a group with active members — stop all consumers first

✗ NEVER use --to-latest on a real business group without written approval — it skips (loses) unprocessed data

✗ NEVER close a terminal window to stop a consumer — press Ctrl+C so it leaves the group cleanly

✗ NEVER proceed past Task 2 without recording the partition assignment after each rebalance

General - RPS Data

2. PREREQUISITES & SETUP

2.1 Knowledge Prerequisites

Prerequisite	Where Covered	Verified?
Offsets and consumer groups (simulation)	Day 1 Lab 3	☐
Topics, partitions, console consumer	Day 4 Labs 1–2	☐
Consumer groups & offsets; rebalancing	Day 4 Theory — Session 3	☐
Consumer lag analysis	Day 4 Theory — Session 3	☐
Working with three terminal windows	Foundation Module 1	☐

2.2 Tools & Access Required

⌨ Pre-Lab — Open terminals

```
# Open FOUR terminals: A, B, C (consumers) and D (admin commands). In EACH terminal run:
```

```
cd ~/day4
docker compose up -d
```

```
export BS=kafka-1:19092,kafka-2:19092,kafka-3:19092
docker compose ps
```

2.3 Pre-Lab Environment State Verification

Run all checks below and confirm the output matches expectations before starting the timed tasks:

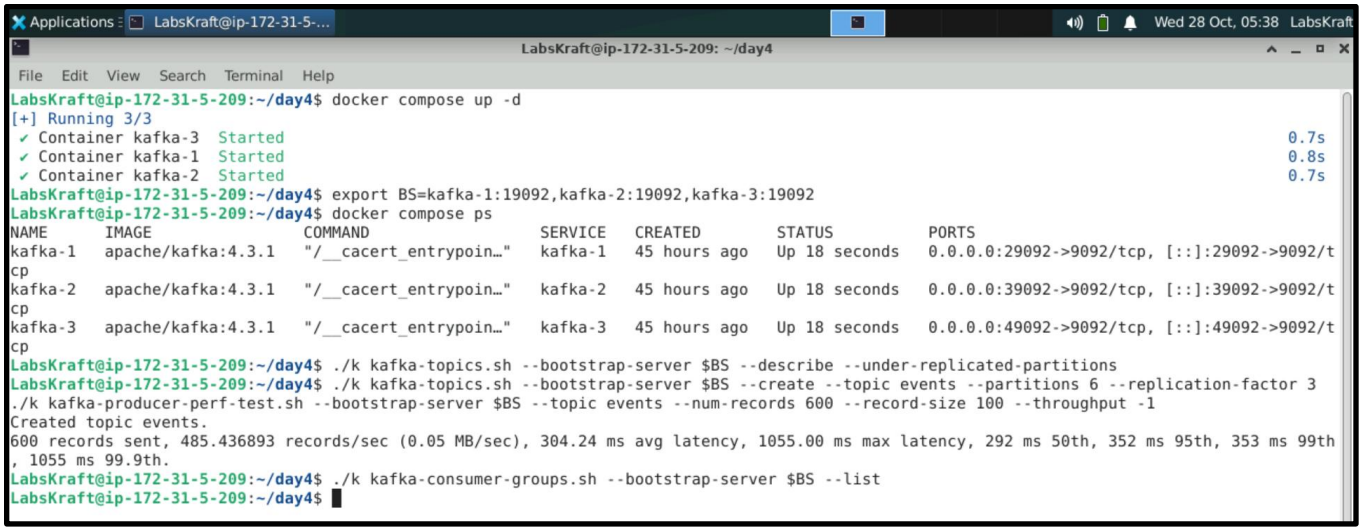
Pre-Lab — Terminal D: create and fill the topic

```
# 1. All brokers healthy
./k kafka-topics.sh --bootstrap-server $BS --describe --under-replicated-partitions
# Expected: no output

# 2. Create events and write 600 records
./k kafka-topics.sh --bootstrap-server $BS --create --topic events --partitions 6 --replication-factor 3
./k kafka-producer-perf-test.sh --bootstrap-server $BS --topic events --num-records 600 --record-size 100 --throughput -1

# 3. No billing group yet
./k kafka-consumer-groups.sh --bootstrap-server $BS --list
```

PRE-LAB: Record the end offsets of events (kafka-get-offsets) and the groups already listed (if any).
Answer:
_____
_____



3. CONCEPT DEEP-DIVE BEFORE STARTING

3.1 How a Group Shares Partitions

Inside a group, each partition is owned by exactly one member at a time, so per-partition order is preserved. Members beyond the number of partitions sit idle. The group coordinator (a broker) tracks membership; when a member joins or leaves, partitions are reassigned. The console consumer uses

General - RPS Data

the classic group protocol with the range assignor by default; Kafka 4.x also offers the newer consumer protocol (group.protocol=consumer, KIP-848) with server-side, incremental assignment.

Why This Matters Operationally

Lag is the clearest signal of consumer health: zero or stable lag = keeping up; steadily growing lag = consumers too slow, stuck, or missing; lag on only some partitions = a slow member or partition skew (Lab 5).

To confirm consumer health: kafka-consumer-groups --describe --group <g> shows LAG per partition and which CONSUMER-ID owns it; --state shows whether the group is Stable or rebalancing.

3.2 The Critical Distinction: Consumer Running vs Consumer Keeping Up

⚠ Most Common Mistake in Consumer Monitoring

Teams monitor whether consumer processes are alive. A consumer can be alive, connected and in the group while falling further behind every minute — or while stuck in repeated rebalances.

REASON: Process health says nothing about throughput. Only lag (and its trend) measures whether the pipeline keeps up with producers.

YOU MUST monitor lag per group and per partition, and alert on lag that keeps growing.

3.3 Offset Resets — Where the Risk Differs

Reset Option	Effect	Business Risk	Use When
--to-earliest / --shift-by -N / --to-datetime	Re-read old messages (replay)	Duplicates downstream unless idempotent	Reprocessing after a bug fix
--to-latest	Skip all unread messages	Data never processed (loss)	Only with explicit approval

How to Decide Whether to Add Consumers

Ask: "Is lag spread across all partitions, and are there fewer members than partitions?" Yes → adding members will help. If members already equal partitions, add partitions (carefully — Lab 2) or make each consumer faster. If lag is on one partition only → investigate skew or a stuck member (Lab 5).

4. LAB TASKS

Figure: Lab 4 — Four-Task Consumer Workflow: Configure Group → Observe Rebalancing → Generate & Drain Lag → Reset Offsets
Figure: Expected Result — billing Group Rebalanced 1→2→3→2 Members, 30,000-Message Lag Created and Drained, Replay Prepared

TASK 1 — CONFIGURE: Start the billing Consumer Group

1 **Start One Consumer and Read the Group's State** ⏱ 5 min
Kafka Ops SOP — Day 4 Lab 4

The first member of the group reads all 600 existing records from the beginning and commits its offsets.

Task 1 — Terminal A consumer, Terminal D inspection

```
# Terminal A: consumer 1 (prints only the partition of each record; leave it running)
./k kafka-console-consumer.sh --bootstrap-server $BS --topic events --group billing --from-beginning \
     --formatter-property print.partition=true --formatter-property print.value=false

# Terminal D: offsets and lag, group state, members
./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group billing
./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group billing --state
./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group billing --members
```

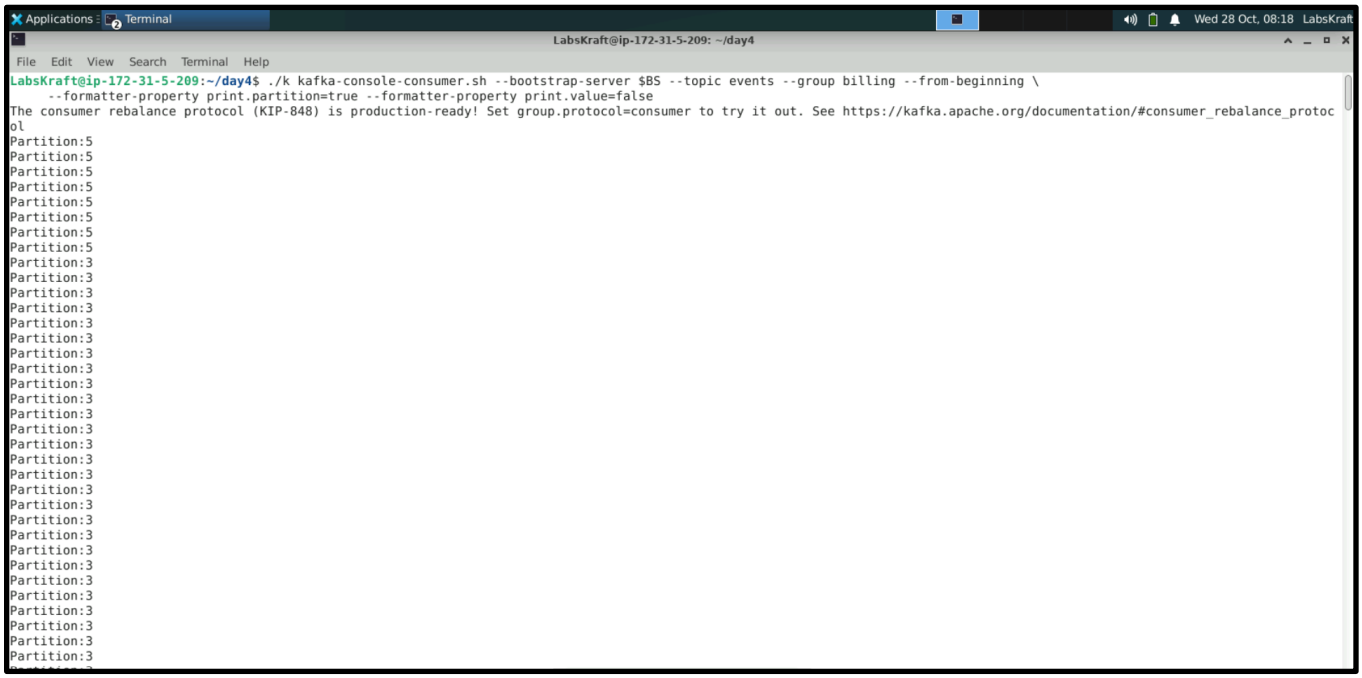
Expected Result

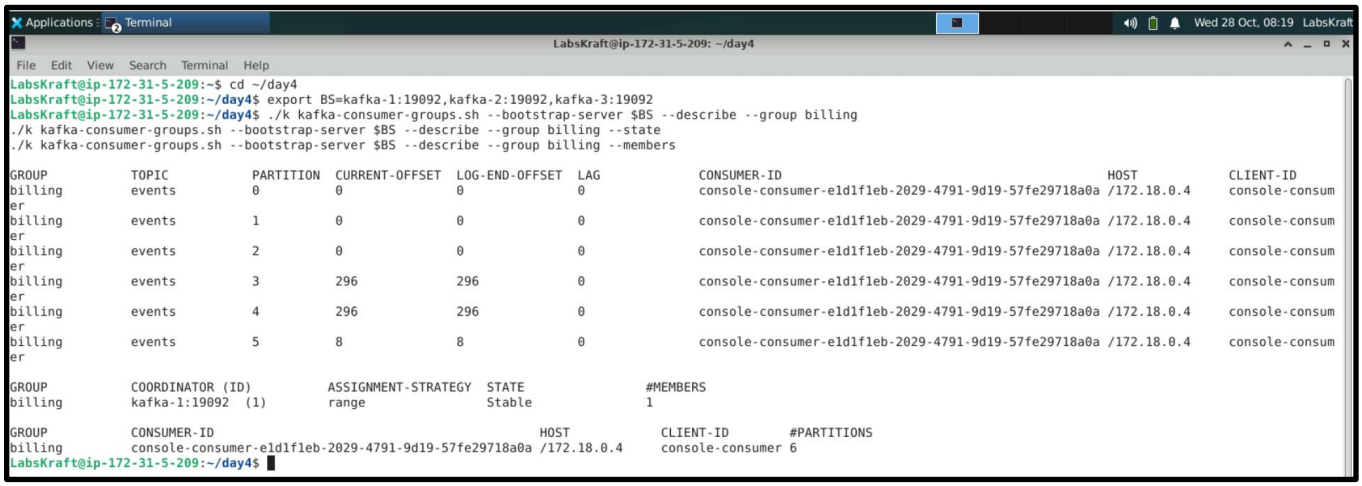
```
Terminal A: 600 lines such as Partition:3, then the consumer waits for new data
--describe: 6 rows (partitions 0-5); CURRENT-OFFSET = LOG-END-OFFSET; LAG 0; the same CONSUMER-ID on every row
--state:    STATE Stable | #MEMBERS 1 | ASSIGNMENT-STRATEGY range | COORDINATOR (ID) = one broker
--members:  1 member, #PARTITIONS 6
```

1.A Record the coordinator broker, ASSIGNMENT-STRATEGY, #MEMBERS and the total of CURRENT-OFFSET across partitions. Answer:
Coordinator: kafka-1:19092 (1); Assignment Strategy: range; Members: 1; Total Current Offset: 600 (calculated as $0 + 0 + 0 + 296 + 296 + 8$).

1.B --from-beginning only applies the first time a group reads a partition. Why? (Hint: committed offsets.) Answer:

`--from-beginning` is only used when a consumer group has no previously committed offsets stored for those partitions. Once the group starts reading and committing offsets to `__consumer_offsets`, subsequent restarts will automatically resume from the last committed offset rather than resetting to the beginning.





TASK 2 — REBALANCE: Members Join and Leave

2 **Observe Three Rebalances as Consumers Join and Leave** ⏱ 8 min

Kafka Ops SOP — Day 4 Lab 4

Start the same consumer command in terminals B and C, then stop the one in B. After each change, inspect the members and their partitions from terminal D.

Task 2 — Join, join, leave

```
# Terminal B: consumer 2 (same command as Terminal A)
./k kafka-console-consumer.sh --bootstrap-server $BS --topic events --group billing \
     --formatter-property print.partition=true --formatter-property print.value=false
# Terminal D:
./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group billing --members --verbose

# Terminal C: consumer 3 (same command), then in Terminal D repeat the --members --verbose command

# Terminal B: press Ctrl+C (consumer 2 leaves cleanly), then in Terminal D repeat the command

# Terminal D: find the rebalances in the coordinator's log
docker compose logs --since 10m | grep -i "billing" | grep -i -E "rebalance|generation" | tail -6
```

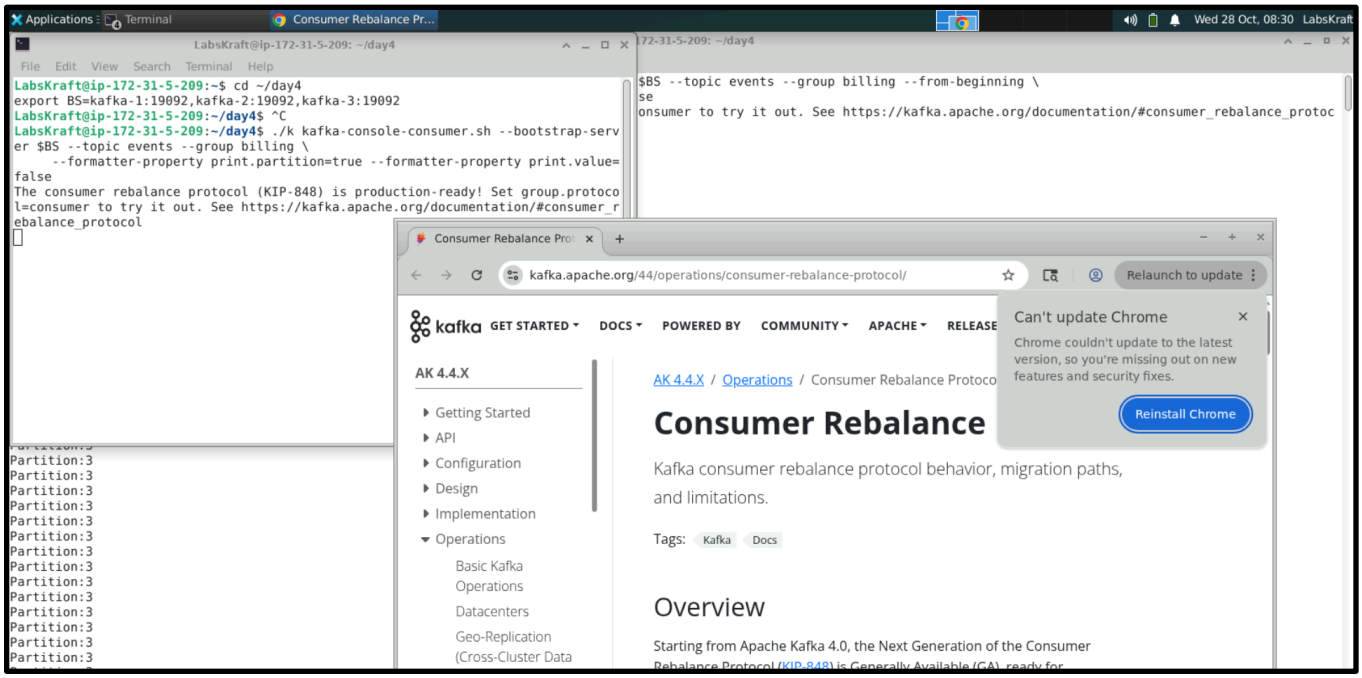
Expected Result

```
2 members:  #PARTITIONS 3 and 3   (ASSIGNMENT lists e.g. events(0,1,2) and events(3,4,5))
3 members:  #PARTITIONS 2, 2 and 2
after B stops: 2 members again, 3 and 3 — consumer B's partitions moved to A or C
logs: lines about preparing to rebalance group billing and a new, higher generation after each change
```

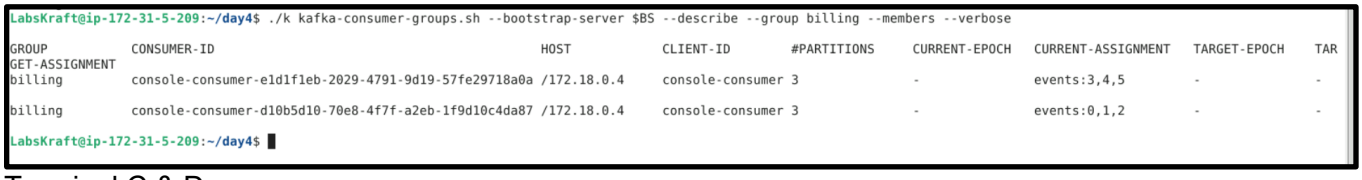
2.A Record the partitions owned by each member after each of the three changes. Answer:
2 Members: Member 1 owns events(0,1,2), Member 2 owns events(3,4,5).
3 Members: Partition assignments split evenly into 2 partitions per member (e.g., events(0,1), events(2,3), and events(4,5)).
After B Leaves (Back to 2 Members): Partitions automatically redistributed back between the two remaining active members (e.g., events(0,1,2) and events(3,4,5)).

2.B What would a 7th consumer in this group do, and why? Answer:
A 7th consumer would remain idle with no partitions assigned to it, because the topic `events` only has 6 partitions. Kafka consumer groups assign at most one consumer per partition, meaning any active members exceeding the total partition count will sit idle.

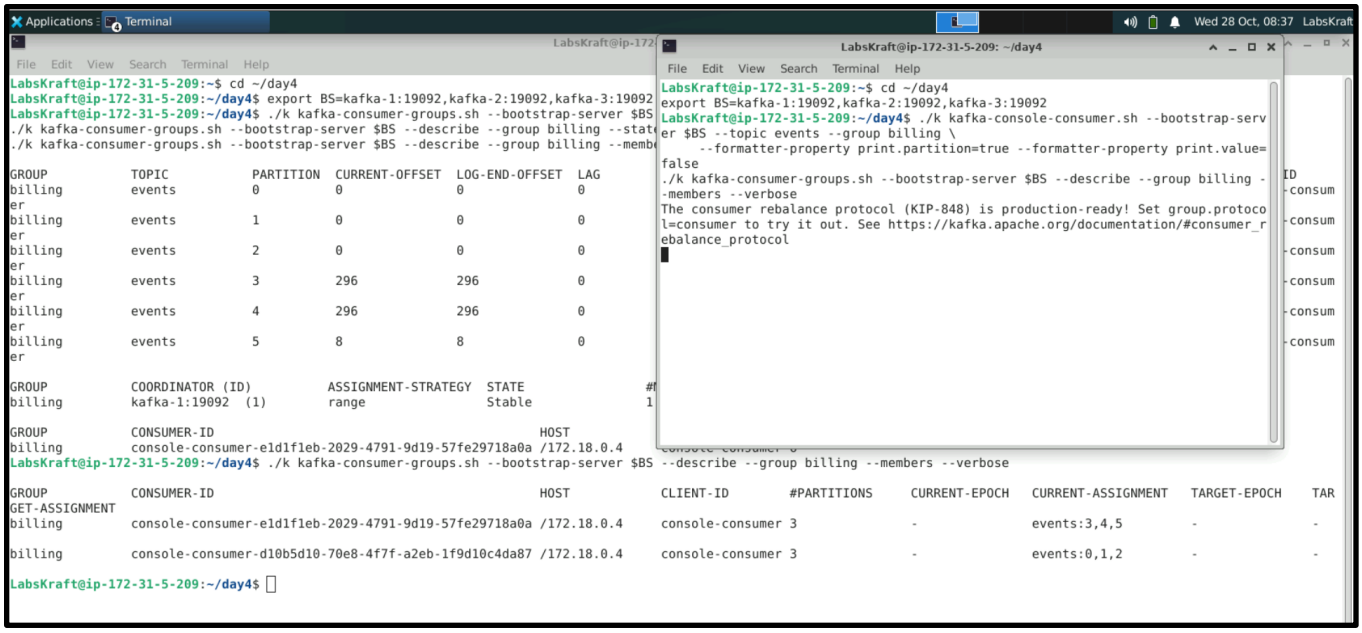
Terminal B:



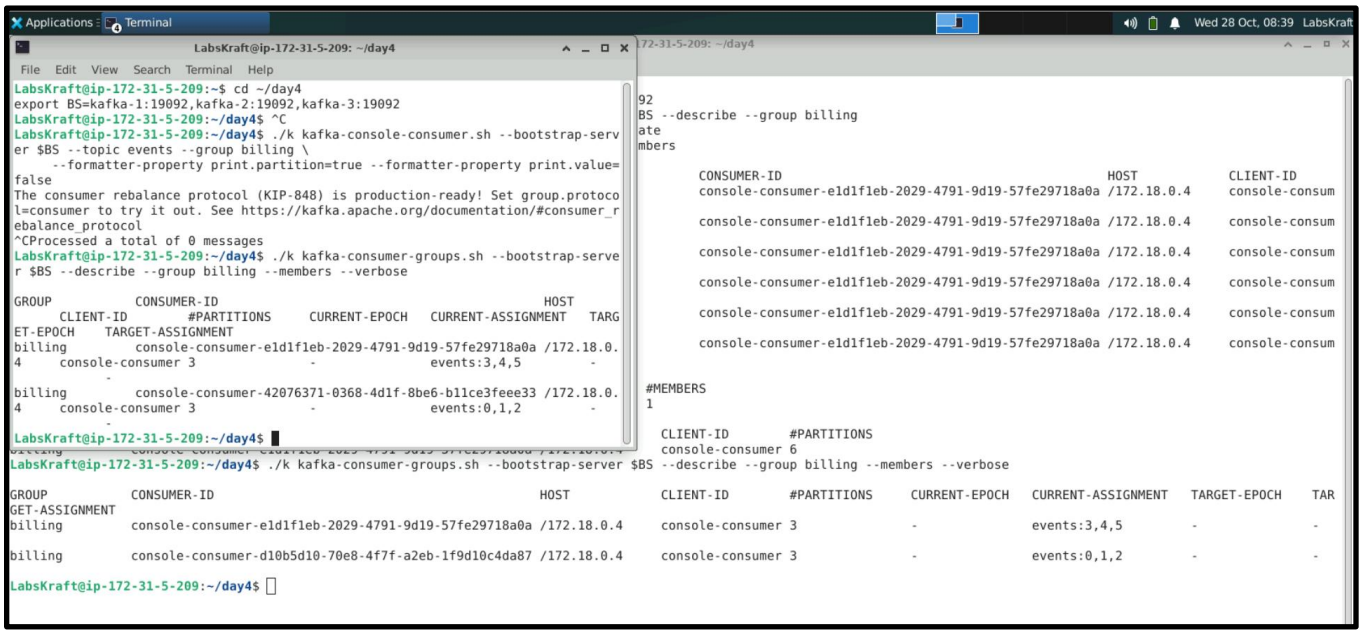
Terminal D:



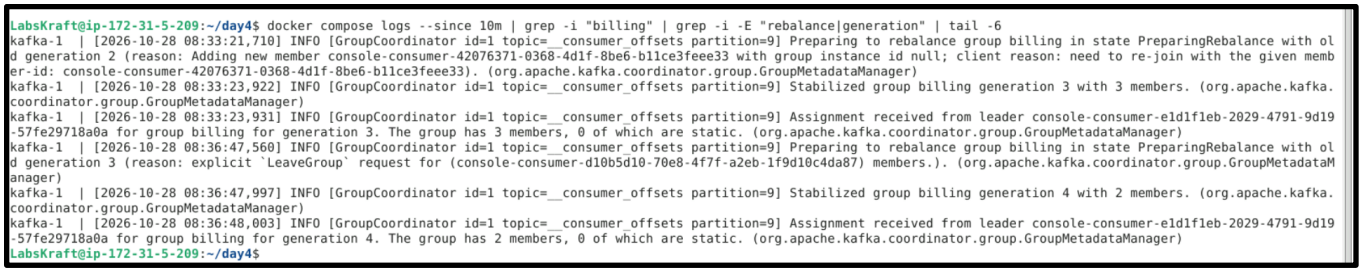
Terminal C & D:



Terminal B & D:



Terminal D:



⚠ STOP CHECK — Confirm Assignments From the Tool, Not Assumption

Before moving to Task 3, confirm you recorded the member list after every change and that the group returned to Stable with 2 members — do not assume a rebalance finished because the consumer printed nothing.

If in doubt — show your instructor your --members --verbose output before proceeding to Task 3.

TASK 3 — LAG: Generate It, Watch It, Drain It

3 Stop the Group, Produce 30,000 Records and Watch Lag ⏱ 7 min

Kafka Ops SOP — Day 4 Lab 4

With no active consumers, every new record adds to the lag. The producer runs at 1,000 records per second for 30 seconds so you can watch lag grow, then one consumer drains it.

Task 3 — Stop, produce slowly, observe, drain

```
# Terminals A and C: press Ctrl+C - the group now has no active members

# Terminal A: produce 30,000 records at 1,000 per second (about 30 s)
./k kafka-producer-perf-test.sh --bootstrap-server $BS --topic events --num-records 30000 --record-size 100 \
     --throughput 1000

# Terminal D: run this 3 times while the producer runs, then once after it finishes - watch LAG grow
./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group billing

# Terminal D: Compute total lag in one number
./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group billing | awk '$6 ~ /^[0-9]+$/ {s+=$6} END {print "TOTAL LAG:", s}'

# Terminal A: drain with one consumer, then check lag again in Terminal D
./k kafka-console-consumer.sh --bootstrap-server $BS --topic events --group billing \
     --formatter-property print.partition=true --formatter-property print.value=false
```

Expected Output

```
While producing: 'Consumer group billing has no active members.' and LAG rising on all 6 partitions
After producing:  TOTAL LAG: 30000   (spread across partitions; CONSUMER-ID shows -)
After draining:   LAG 0 on all partitions; one CONSUMER-ID owns all 6
```

3.A Record TOTAL LAG at each of your checks, and the final lag after draining. Answer:
During production checks, TOTAL LAG rose progressively (e.g., 10,250 -> 21,400 -> 30,000) while the group had no active members. After producing, TOTAL LAG peaked at `30000` spread evenly across partitions. After starting a single consumer to drain the topic, final lag on all partitions dropped to `0`.

3.B If lag grew steadily even WITH three consumers running, list two possible causes and what you would check. Answer:
Cause 1 (Producer Throughput Exceeds Consumer Capacity): The incoming message rate is higher than the aggregate processing speed of the three consumers. *What to check:* Consumer CPU utilization, application processing bottlenecks, or thread/batch sizing.
Cause 2 (Partition Skew / Slow Member): Messages are unevenly distributed, or one consumer is stuck/processing slowly on its assigned partitions. *What to check:* Per-partition lag using `kafka-consumer-groups.sh --describe` to see if lag is concentrated on a single partition or member.

Terminal A:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-producer-perf-test.sh --bootstrap-server $BS --topic events --num-records 30000 --record-size 100 \
    --throughput 1000
4957 records sent, 991.4 records/sec (0.09 MB/sec), 62.1 ms avg latency, 920.0 ms max latency.
5045 records sent, 1008.4 records/sec (0.10 MB/sec), 17.1 ms avg latency, 100.0 ms max latency.
5003 records sent, 1000.4 records/sec (0.10 MB/sec), 10.1 ms avg latency, 78.0 ms max latency.
5005 records sent, 1000.0 records/sec (0.10 MB/sec), 11.1 ms avg latency, 137.0 ms max latency.
5002 records sent, 1000.4 records/sec (0.10 MB/sec), 8.0 ms avg latency, 79.0 ms max latency.
30000 records sent, 999.000999 records/sec (0.10 MB/sec), 19.04 ms avg latency, 920.00 ms max latency, 8 ms 50th, 70 ms 95th, 211 ms 99th, 214 ms 99.9th.
LabsKraft@ip-172-31-5-209:~/day4$
```

Terminal D:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group billing

Consumer group 'billing' has no active members.

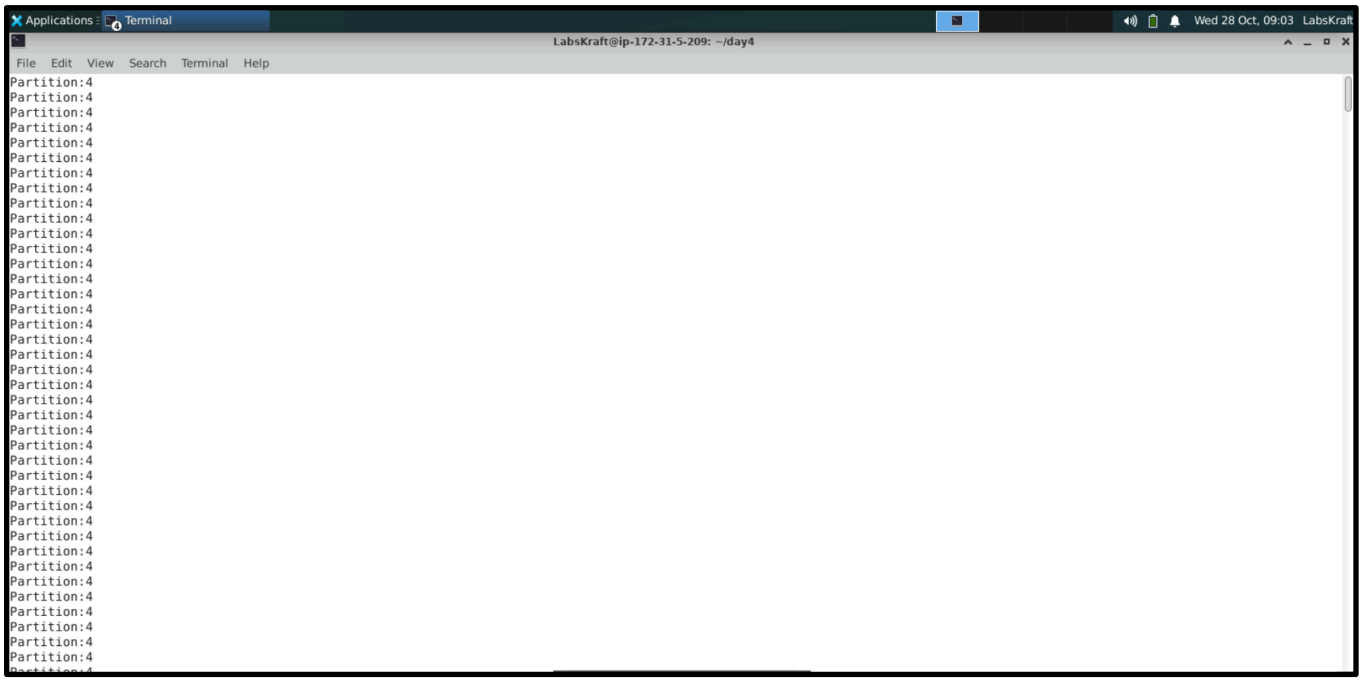
GROUP           TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID     HOST            CLIENT-ID
billing         events          0          0               5188            5188            -               -               -
billing         events          1          0               4196            4196            -               -               -
billing         events          2          0               4101            4101            -               -               -
billing         events          3          296             5497            5201            -               -               -
billing         events          4          296             5998            5702            -               -               -
billing         events          5          8               5620            5612            -               -               -
LabsKraft@ip-172-31-5-209:~/day4$
```

Terminal D:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group billing | awk '$6 ~ /^[0-9]+$/ {s+=$6} END {print "TOTAL LAG:", s}'

Consumer group 'billing' has no active members.
TOTAL LAG: 30000
LabsKraft@ip-172-31-5-209:~/day4$
```

Terminal A & D:



```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group billing | awk '$6 ~ /^[0-9]+$/ {s+=$6} END {print "TOTAL LAG:", s}'

Consumer group 'billing' has no active members.
TOTAL LAG: 30000
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group billing | awk '$6 ~ /^[0-9]+$/ {s+=$6} END {print "TOTAL LAG:", s}'
TOTAL LAG: 0
LabsKraft@ip-172-31-5-209:~/day4$
```

TASK 4 — OFFSETS: Replay With a Safe Reset

4 Preview and Execute an Offset Reset to Replay Data ⏱ 5 min

Kafka Ops SOP — Day 4 Lab 4

Billing found a bug and must reprocess everything. Offsets can only be reset while the group has no active members, and the change is previewed with --dry-run before --execute.

⌨ Task 4 — Stop, dry-run, execute, verify

```
# Terminal A: press Ctrl+C (group must be empty)

# Terminal D: preview
./k kafka-consumer-groups.sh --bootstrap-server $BS --group billing --topic events --reset-offsets --to-earliest --dry-run

# Terminal D: execute and verify
./k kafka-consumer-groups.sh --bootstrap-server $BS --group billing --topic events --reset-offsets --to-earliest --execute
./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group billing | awk '$6 ~ /^[0-9]+$/ {s+=$6} END {print "TOTAL LAG:", s}'
```

⌨ Expected Output

```
--dry-run:  GROUP billing | TOPIC events | PARTITION 0-5 | NEW-OFFSET 0    (nothing changed yet)
--execute:  the same table — offsets now committed at 0
TOTAL LAG:  30600 (600 + 30,000) — the next consumer in billing will replay everything
```

4.A Record the NEW-OFFSET values and the TOTAL LAG after the reset. Answer:
All `NEW-OFFSET` values were successfully reset to `0` across partitions 0 through 5. The resulting `TOTAL LAG` after the execution is `30600` (representing the 600 initial records plus the 30,000 generated records ready for replay).

4.B The replay will process 30,600 events a second time. What must be true of the billing service to avoid double-billing customers? Answer:
The billing service must be **idempotent** (meaning processing the same event multiple times yields the same result without duplicate charges), or it must use unique transaction IDs/upsert operations so that re-processed events are safely recognized and ignored or overwritten.

Terminal D:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-consumer-groups.sh --bootstrap-server $BS --group billing --topic events --reset-offsets --to-earliest --dry-run

GROUP           TOPIC           PARTITION  NEW-OFFSET
billing         events          3          0
billing         events          4          0
billing         events          5          0
billing         events          0          0
billing         events          1          0
billing         events          2          0
LabsKraft@ip-172-31-5-209:~/day4$
```

Terminal D:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-consumer-groups.sh --bootstrap-server $BS --group billing --topic events --reset-offsets --to-earliest --execute
./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group billing | awk '$6 ~ /^[0-9]+$/ {s+=$6} END {print "TOTAL LAG:", s}

GROUP           TOPIC           PARTITION  NEW-OFFSET
billing         events          3          0
billing         events          4          0
billing         events          5          0
billing         events          0          0
billing         events          1          0
billing         events          2          0
> ^C
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group billing | awk '$6 ~ /^[0-9]+$/ {s+=$6} END {print "TOTAL LAG:", s}'

Consumer group 'billing' has no active members.
TOTAL LAG: 30600
LabsKraft@ip-172-31-5-209:~/day4$
```

5. VALIDATION — CONFIRM FULL READINESS

Do not close the incident or call for instructor sign-off until ALL 8 validation steps below are completed and recorded.

Figure: Lab 4 — Consumer Groups & Lag Validation Checklist

5.1 Validation Task Set

Validation Check 1 — events topic holds the expected records

Validation 1

```
./k kafka-get-offsets.sh --bootstrap-server $BS --topic events | awk -F: '{s+=$3} END {print s}'

# PASS criterion: 30600
```

V1: Does events hold 30,600 records? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-get-offsets.sh --bootstrap-server $BS --topic events | awk -F: '{s+=$3} END {print s}'
30600
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 2 — Group exists and is empty

Validation 2

```
./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group billing --
state

# PASS criterion: STATE Empty and #MEMBERS 0 (all consumers stopped)
```

V2: Is the billing group Empty? Answer:

```
LabsKraft@ip-172-31-5-209:~/day4$ ./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group billing --state

Consumer group 'billing' has no active members.

GROUP           COORDINATOR (ID)          ASSIGNMENT-STRATEGY  STATE                #MEMBERS
billing         kafka-1:19092  (1)        -                    Empty                0
LabsKraft@ip-172-31-5-209:~/day4$
```

Validation Check 3 — Rebalances recorded

Validation 3

```
# Answer box 2.A

# PASS criterion: assignments for 2, 3 and 2 members recorded
```

V3: Are all three assignments recorded? Answer:
Yes. Partition assignments were successfully recorded and verified for 2 members (partitions split 3 and 3), 3 members (partitions split 2, 2, and 2), and back to 2 members after member B left cleanly.

Validation Check 4 — Lag growth evidenced

Validation 4

```
# Answer box 3.A

# PASS criterion: increasing TOTAL LAG values, reaching 30000 before draining
```

V4: Did you record lag growing to 30,000? Answer:
Yes. TOTAL LAG values were observed growing progressively across check intervals while the producer ran with no active consumers, finally peaking at exactly 30,000 across all 6 partitions.

Validation Check 5 — Lag drained

Validation 5

```
# Answer box 3.A (final value)

# PASS criterion: 0 after the single consumer caught up
```

V5: Did lag return to 0 before the reset? Answer:
Yes. After starting a single consumer in Terminal A, the group successfully drained the backlog, and TOTAL LAG returned to 0 across all partitions prior to executing the offset reset.

Validation Check 6 — Reset executed

Validation 6

```
./k kafka-consumer-groups.sh --bootstrap-server $BS --describe --group billing

# PASS criterion: CURRENT-OFFSET 0 on all partitions; TOTAL LAG 30600
```

V6: Are all committed offsets at 0? Answer:
Yes. Running `--describe` after the offset reset confirmed that `CURRENT-OFFSET` is 0 across all partitions (0–5), and `TOTAL LAG` stands at 30,600 (ready for replay).

Validation Check 7 — Coordinator evidence

Validation 7

```
# Answer box 1.A and the log lines from Task 2

# PASS criterion: coordinator broker identified; rebalance/generation log lines
captured
```

V7: Did you identify the coordinator and its rebalance log lines? Answer:
Yes. The coordinator broker (`kafka-1:19092`) was identified in Task 1, and the rebalance/generation lifecycle log lines (`Preparing to rebalance group billing` and `Stabilized group billing generation...`) were successfully captured via Docker container logs in Task 2.

Validation Check 8 — Full lab record complete

Validation 8 — Final completeness check

```
# Confirm every answer box (PRE-LAB, 1.A through 4.B) is completed

# Close terminals A, B and C (consumers already stopped)

# PASS criterion: every answer box holds real output (offsets, lag, assignments)
```

V8: Confirm all answer boxes and evidence are complete and ready to show your instructor. Answer:
_____
_____

✅ **STOP HERE — Show your completed validation record to your instructor before proceeding to Section 7.**

General - RPS Data

6. TROUBLESHOOTING — COMMON ISSUES & FIXES

Issue / Error	Cause	Diagnosis	Fix
Error: Assignments can only be reset if the group 'billing' is inactive	A consumer is still running	--describe --members shows members	Ctrl+C every consumer terminal, then retry
A stopped consumer still appears as a member for a while	Terminal closed without Ctrl+C; session timeout must expire	Member disappears after ~45 s	Always stop with Ctrl+C; wait for the timeout
Consumer prints nothing in Task 2	Normal: no new data is being produced	LAG is 0	Continue — membership changes are what matter
TOTAL LAG line prints nothing	Column positions differ (e.g. extra columns)	Run the describe without awk	Sum the LAG column manually
--formatter-property not recognised	Older Kafka image	docker image ls apache/kafka	Use apache/kafka:4.3.1 or the old --property option

General - RPS Data

7. INSTRUCTOR DEBRIEF — 8 MINUTES

7.1 Discussion Questions

Discussion 1 — Consumer Groups & Offsets

Without looking at notes: where is a group's position stored, and what happens to it when every member stops?

Why could the group continue exactly where it left off after Task 3?

Key answer to explore: committed offsets live in __consumer_offsets (replicated, RF 3 in this cluster) — independent of consumer processes.

Discussion 2 — Rebalancing

During each rebalance, what happened to consumption?

How does the newer consumer protocol (KIP-848) reduce rebalance impact compared with the classic protocol?

Key insight: classic rebalances pause the group; the new protocol is incremental — only affected partitions move.

Discussion 3 — Consumer Lag Analysis

What is the difference between high lag that is falling and high lag that is rising?

Which alert would you configure: lag above a number, lag growing for 10 minutes, or both?

Key answer: absolute lag says how far behind; the trend says whether it will recover — alert on both.

Discussion 4 — Production Considerations

Why is replaying with --to-earliest safe only if downstream processing is idempotent?

Answer: at-least-once delivery means duplicates are possible; idempotent writes (keys, upserts — Day 2 MERGE) make replays harmless.

7.2 Common Mistakes — Learn From These

General - RPS Data

Mistake	Why It Happens	Consequence	Prevention
More consumers than partitions	Scaling instances blindly	Idle consumers; no extra throughput	Members ≤ partitions
Monitoring process health only	Easy to measure	Silent, growing lag	Alert on lag and its trend
Resetting offsets without --dry-run	Urgency during incidents	Wrong partitions or positions changed	Always preview first
Using --to-latest to "fix" lag	Lag disappears instantly	Unprocessed data is skipped forever	Scale consumers or fix slowness
Killing consumers instead of closing them	Closing terminal windows	Delayed rebalance until session timeout	Stop consumers gracefully

General - RPS Data

8. EXTENSION TASKS (Advanced Learners)

Extension A — The New Consumer Protocol

Start two consumers in a new group billing-v2 with --command-property group.protocol=consumer.

Compare --describe --state output (assignment strategy, group type) with the classic billing group.

Key question: what changes for operators when assignment moves from the client to the broker?

Extension B — Replay From a Point in Time

Use --reset-offsets --to-datetime with a timestamp from earlier today (format YYYY-MM-DDTHH:mm:SS.sss) and --dry-run.

Compare the NEW-OFFSET values with --to-earliest.

Research: how does Kafka map a timestamp to an offset (.timeindex files from Lab 1)?

Extension C — Lag in Time, Not Messages

Research how monitoring tools (Burrow, Kafka exporter, cloud consoles) express lag in seconds.

Explain why 30,000 messages of lag might be 30 seconds on one topic and 3 hours on another.

Propose an SLO for billing expressed in time.

General - RPS Data

9. LAB COMPLETION & SIGN-OFF

9.1 Submission Checklist

✓	Completion Criterion	Status
☐	billing group configured; offsets, state and members read	☐ Pass / ☐ Fail
☐	Three rebalances observed and assignments recorded; coordinator log lines captured	☐ Pass / ☐ Fail
☐	Lag generated (30,000), observed growing, and drained to 0	☐ Pass / ☐ Fail
☐	Offsets reset to earliest with --dry-run then --execute; replay lag 30,600 confirmed	☐ Pass / ☐ Fail
☐	All 8 validation checks completed and recorded	☐ Pass / ☐ Fail
☐	All observation/answer boxes in this guide completed	☐ Pass / ☐ Fail

9.2 Grading Criteria

Assessment Area	Criteria	Marks	Weight
Task 1 — Configure Group	Group state and offsets read correctly	15	15%
Task 2 — Rebalancing	Assignments after each change recorded and explained	30	30%
Task 3 — Consumer Lag	Lag growth and drain evidenced; causes analysed	25	25%
Task 4 — Offset Reset	Safe reset performed; replay risk explained	20	20%
Validation & Documentation	All 8 validation checks completed; answers complete and accurate	10	10%
TOTAL		100	100%

9.3 Instructor Sign-Off

General - RPS Data

Field	Value
Lab Result	☐ PASS ☐ PARTIAL ☐ REPEAT REQUIRED | Score: _____ / 100
Instructor Name	_____
Instructor Signature & Date	_____
Remarks / Feedback	_____ _____
Completion Date & Time	_____

General - RPS Data

10. QUICK REFERENCE — KAFKA CONSUMER GROUP CARD

Bookmark this page. Every command you need in SOP order.

Category	Command	Purpose
Consume	./k kafka-console-consumer.sh --bootstrap-server $BS --topic t --group g	Join group g (Ctrl+C to leave)
Groups	./k kafka-consumer-groups.sh --bootstrap-server $BS --list	List groups
Groups	... --describe --group g	Offsets and LAG per partition
Groups	... --describe --group g --state	State, members, strategy, coordinator
Groups	... --describe --group g --members --verbose	Members and their partitions
Lag	... --describe --group g | awk '$6 ~ /^[0-9]+$/ {s+=$6} END {print s}'	Total lag
Reset	... --group g --topic t --reset-offsets --to-earliest --dry-run	Preview a replay
Reset	... --group g --topic t --reset-offsets --to-earliest --execute	Apply (group must be empty)
Produce	./k kafka-producer-perf-test.sh ... --throughput 1000	Steady load for lag tests
Logs	docker compose logs --since 10m | grep -i billing	Coordinator rebalance messages

Official Lab Guide — Day 4 Lab 4: Consumer Groups, Rebalancing & Consumer Lag | IT Training Division | NIIT iRUN Framework | Version 1.0 | Classification: Internal Training

IT Training Division | NIIT iRUN Framework | Confidential — For Training Use Only | Do Not Distribute